Algorithms for NP-Hard Problems

Summary note, Lectured by Eunjung Kim, KAIST

Hojune Lee

School of Computing, KAIST

Contents

1 Parameterized Algorithm **3**

2 Branching Algorithms **5**

2.1 $\mathcal{O}(2^k)$-time Algorithm for Vertex Cover 5

2.2 $\mathcal{O}((3k)^k)$-time Algorithm for Feedback Vertex Set 6

2.3 $\mathcal{O}(4^{k-lp^*(G)})$-time Algorithm for Vertex Cover 8

3 Kernelization **14**

3.1 $\mathcal{O}(k^2)$-Vertex Kernel 14

3.2 LP-based kernelization for Vertex Cover 16

3.3 $3k$-vertex kernel using Crown Decomposition 17

3.4 $\mathcal{O}(k^2)$-Vertex Kernel for Feedback Vertex Set 21

4 Iterative Compression **27**

4.1 $\mathcal{O}(5^k)$-time algorithm for Feedback Vertex Set 27

5 Randomized Algorithms **30**

6 Tree Decomposition **31**

Chapter 1

Parameterized Algorithm

Parameterization

The *parameterization* is a map defined by

$$\kappa : \Sigma^* \to \mathbb{N}_0$$

With this definition, *parameterized problem* is an ordered pair decision problem for language $\mathfrak{L} \subseteq \Sigma^*$ and parameterization κ, $(\mathfrak{L}, \kappa)$. Since κ is a map, we can take any well-defined function for κ. So, we can claim that for wrong encoded instance in Σ^*, they goes to some fixed value, e.g. 0.
Here, it is important that why we take this definition. I'll write the intuition later, after introducing the example.

Vertex Cover

All problems are defined in red-colored box. First problem is Vertex Cover. For a graph $G = (V, E)$ and $k \in \mathbb{N}_0$, *vertex cover* is a set $S \subseteq V$ such that $\forall uv \in E, u \in S \vee v \in S$. i.e. All edges in E are covered by S.
Here, Vertex Cover is a decision problem of following encoded language $\mathfrak{L}$:

$$\mathfrak{L}_{VC} = \{(G, k) | G = (V, E), k \in \mathbb{N}_0, G \text{ has vertex cover whose size at most } k\}$$

Since $(G, k) \in \Sigma^*$ is a encoded string, they are together. Means that, when we describe the input size $|x|$ of algorithm, it doesn't contain the seperated meaning of size of G and magnitude of k. So we parameterization κ here, to extract the meaning of k.

$$\kappa : (G, k) \mapsto k$$

As we'll see later, another information also can be extracted :

$$\kappa : (G, k) \mapsto k - lp^*(G)$$

This is conceptual meaning of parameterization of a problem.

Fixed Parameter Tractable

A parameterized problem $(\mathfrak{L}, \kappa)$ is *fixed parameter tractable* if there is an algorithm such that for every $x \in \Sigma^*$, decides whether $x \in \mathfrak{L}$ or not in time $f(\kappa(|x|)) \cdot |x|^{\mathcal{O}(1)}$

Chapter 2

Branching Algorithms

2.1 $\mathcal{O}(2^k)$-time Algorithm for Vertex Cover

Algorithm 1 $\mathcal{O}(2^k)$-time Algorithm for Vertex Cover

1: **procedure VC**(G, k)

Input: $G = (V, E)$ is a graph, $k \in \mathbb{N}_0$

Output: Decide $(G, k) \in \mathfrak{L}_{VC}$ or not

2: **if** $|E| = 0$ **then return** Yes

3: **else if** $k = 0$ **then return** No

4: **else**

5: $uv \leftarrow Pick(E)$ ▷ Pick any edge

6: **return VC**$(G - u, k - 1) \vee$ **VC**$(G - v, k - 1)$

This is very fundamental algorithm for Vertex Cover. We can prove following:

Algorithm 1

Algorithm 1 is *correct*(i.e., decides whether $(G, k) \in \mathfrak{L}_{VC}$ or not correctly) in time $\mathcal{O}(2^k \cdot poly(n))$

Proof. We can say that algorithm 1 is correct means that

$$\mathbf{VC}(G, k) = \text{Yes} \iff (G, k) \in \mathfrak{L}_{VC}$$

We'll prove via mathematical induction on $k \in \mathbb{N}_0$. If $k = 0$, it is trivial. Then assume that for any $G = (V, E)$, $VC(G, i) = \text{Yes} \iff (G, i) \in \mathfrak{L}_{VC}, 0 \leq i < k$. And one easy fact is that, if S is a vertex cover of G, and $u \in S$, then $S \setminus \{u\}$ is a vertex cover of $G - u$.
($\Leftarrow$) Let $(G, k) \in fL_{VC}$. Then, exists a vertex cover S of G such that $|S| \leq k$. Suppose that given instance is non-trivial, so reach at line 4. Then pick $\forall uv \in E$, and W.L.O.G., $u \in S$. Then by fact, $S \setminus \{u\}$ is a vertex cover of $G - u$ whose size is at most $k - 1$. Then, we can determine that $\mathbf{VC}(G - u, k - 1) = \text{Yes}$ by hypothesis.
($\Rightarrow$) Suppose that $\mathbf{VC}(G, k) = \text{Yes}$. Then for non-trivial cases, W.L.O.G., $\mathbf{VC}(G - u, k -$

1) is Yes. It implies that $(G-u, k-1) \in \mathfrak{L}_{VC}$. Then, let S' be the vertex cover of $G-u$ whose size is at most $k-1$. Then, let $S = S' \cup \{u\}$. It is clearly a vertex cover of G whose size is at most k. So $(G, k) \in \mathfrak{L}_{VC}$.

Now, for the running time. It is obvious that line 2,3,5 effects polynomial time w.r.t. input size. And there are at most k-depth of binary recursion, so running time is $\mathcal{O}(2^k \cdot poly(n))$.

□

2.2 $\mathcal{O}((3k)^k)$-time Algorithm for Feedback Vertex Set

Now we define next problem to solve, which is called Feedback Vertex Set.

Feedback Vertex Set

For any graph $G = (V, E)$, a *feed-back vertex set* of G is a set $S \subseteq V$ such that

$$\forall C \in Cycle(G), S \cap C \neq \emptyset$$

i.e., set of vertices that intersect with all of cycles in G. So, we can define decision problem Feedback Vertex Set by

$$\mathfrak{L}_{FVS} = \{(G, k) | G = (V, E), k \in \mathbb{N}_0, G \text{ has FVS whose size at most } k\}$$

Algorithm 2 $\mathcal{O}((3k)^k)$-time Algorithm for Feedback Vertex Set

```
1: procedure FVS(G, k)
2:     if |E| = 0 then return Yes
3:     if k = 0 then return No
4:     while ∃v ∈ V(G), d_G(v) ≤ 2 do
5:         if d_G(v) ≤ 1 then
6:             G ← G − v
7:         else if |N_G(v) \ {v}| = 2 then
8:             {x, y} ← N_G(v) \ {v}
9:             G ← G − v + xy                          ▷ Allow multigraphs
10:        else
11:            G ← G \ N_G(v)
12:            k ← k − 1          ▷ Assume that we choose the adjacent vertex of v
13:    {v_n, ··· , v_1} ← Sort(V(G))                   ▷ Note: Decreasing Order
14:    return ⋁_{i=1}^{3k} FVS(G − v_i, k − 1)
```

Algorithm 2

Algorithm 2 is correct for FEEDBACK VERTEX SET in time $\mathcal{O}((3k)^k \cdot poly(n))$

Lemma 2.1. *For any feed-back vertex set X of $G = (V, E)$ holds that*

$$\sum_{x \in X} (d_G(x) - 1) \geq |E| - |V| + 1$$

Proof. One easily obtained fact is that, $G[V \setminus X]$ is a forest. Means that,

$$|E(G[V \setminus X])| \leq |V(G[V \setminus X])| - 1 = |V| - |X| - 1$$

However,

$$|E| = \underbrace{|E(G[X])| + |E(X, V \setminus X)|}_{\leq \sum_{x \in X} d_G(x)} + |E(G[V \setminus X])|$$

So,

$$\sum_{x \in X} d_G(x) \geq |E| - |E(G[V \setminus X])| \geq |E| - |V| + |X| + 1$$

Implies that

$$\sum_{x \in X} (d_G(x) - 1) \geq |E| - |V| + 1$$

□

Lemma 2.2. *Let $G = (V, E)$ such that $\delta(G) \geq 3$. Let $V = \{v_1, v_2, \cdots, v_n\}$ where*

$$d_G(v_1) \geq d_G(v_2) \geq \cdots \geq d_G(v_n)$$

Then, for any feedback vertex set X of G,

$$|X| \leq k \implies X \cap \{v_1, \cdots, v_{3k}\} \neq \emptyset$$

Proof. Suppose not. By hand-shake lemma,

$$\sum_{v \in V} d_G(v) = 2|E| \geq 3|V| \qquad \because \delta(G) \geq 3$$

And since $X \cap \{v_1, \cdots, v_{3k}\} = \emptyset$, We can obtain following from above lemma:

$$\sum_{i=1}^{3k} (d_G(v_i) - 1) \geq 3 \sum_{x \in X} (d_G(x) - 1) \geq 3|E| - 3|V| + 3$$

$$\sum_{i=3k+1}^{n} (d_G(v_i) - 1) \geq \sum_{x \in X} (d_G(x) - 1) \geq |E| - |V| + 1$$

implies that,

$$\sum_{v \in V(G)} (d_G(v) - 1) = 2|E| - |V| \geq 4|E| - 4|V| + 4$$

It deduce

$$3|V| > 2|E|$$

which contradict to first line. □

With second lemma, one can show that Algorithm 2 is correct. And since it generates at most $3k$ branches for each step, and the maximal depth is k, so the running time will be $\mathcal{O}((3k)^k \cdot poly(n))$

2.3 $\mathcal{O}(4^{k-lp^*(G)})$-time Algorithm for Vertex Cover

In this section, we review the VERTEX COVER with another parameterization, which is

$$\kappa : (G, k) \mapsto k - lp^*(G)$$

Here, $lp^*(G)$ is a solution of another problem, which called linear program version of vertex cover.

Linear Program for Vertex Cover

For $G = (V, E)$, linear program for vertex cover is called LPVC, which defined as following.

$$\min \sum_{u \in V(G)} x_u$$

where

$$\forall uv \in E, x_u + x_v \geq 1, \qquad \forall u \in V, x_u \geq 0 \tag{$\star$}$$

holds. Let's call above condition by $\star$. And we denote $lp^*(G)$ be the optimal solution of LPVC.

One fact is that,

$$\text{size of optimal vertex cover of } G \geq lp^*(G)$$

This is because that, optimal vertex cover is very special case of LPVC. It can be easily obtain, since every vertex cover satisfy the condition $\star$. So, this automatically implies that for given (G, k), if $k < lp^*(G)$, (G, k) is No instance of VERTEX COVER.

Now, we'll explore the algorithm which related to these LP problems. Before move on, we'll take a following theorem without proof.

Linear Program is P

Linear Program can be solved in polynomial time. So that, LPVC also solved in polynomial time.

And now we'll define half-integral solution for LPVC.

Half Integral solution for LPVC

We call that the solution x^* for LPVC is *half-integral* when

$$\forall u \in V(G), x_u^* \in \{0, \frac{1}{2}, 1\}$$

With the above fact, we can conduct following decomposition in polynomial time.

Crown Decomposition

Let x^* be the solution of LPVC. Then we can partition $V(G)$ into following 3 sets :

$$C = \{u \in V(G) | x_u^* < 0.5\}$$

$$H = \{u \in V(G) | x_u^* > 0.5\}$$

$$R = \{u \in V(G) | x_u^* = 0.5\}$$

This is called *crown decomposition* of G. This decomposition can be obtained in polynomial time.

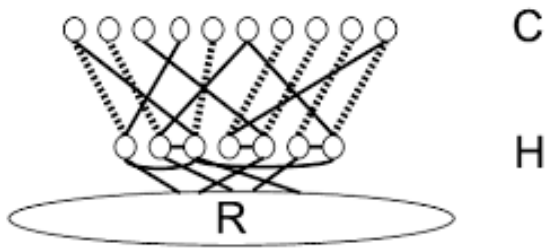

Figure 2.1: A graph illustrating the crown decomposition.

Now, we are going to argue many properties on crown decomposition.

Lemma 2.1. *For every $H' \subseteq H$, $|N_G(H') \cap C| \geq |H'|$*

Proof. Actually, this is quite intuitive. If not, that situation is very un-natural to reduce the total-sum.

Suppose not, i.e., there exists $H' \subseteq H$ such that $|N_G(H') \cap C| < |H'|$. Now idea is very simple, let's construct more cheap solution from here! We can define following value $\epsilon > 0$

$$\epsilon = \min_{h \in H'} \{x_h^* - 0.5\} > 0$$

And then, we'll construct more cheap solution by

$$\forall u \in V(G), x'_u = \begin{cases} x_u^* + \epsilon & u \in N_G(H') \cap C \\ x_u^* - \epsilon & u \in H' \\ x_u^* & \text{otherwise} \end{cases}$$

x' is feasible LP solution since all our reponse to check is for incident parts of H', but it preserves H' to C sum, and since x' have same crown decomposition, H' to H and H' to R also resolved. However, as $|N_G(H') \cap C| < |H'|$, the total cost $|x'|$ is strictly small than $|x^*|$.

$$\sum_{u \in V(G)} x'_u = \left(\sum_{u \in V(G)} x_u^* \right) - \epsilon(|H'| - |N_G(H') \cap C|) < \sum_{u \in V(G)} x_u^*$$

This contradicts to x^* is an optimal solution.

□

Lemma 2.2. *Let (R, H, C) be a crown decomposition of G w.r.t. optimal solution x^* of* LPVC. *Then, following solution is also a optimal solution of* LPVC.

$$x'_u = \begin{cases} 0.5 & u \in R \\ 1 & u \in H \\ 0 & u \in C \end{cases}$$

Proof. Let $\mathcal{X}$ be the set of all optimal solution of LPVC such that yields crown decomposition (R, H, C). And for each $x^* \in \mathcal{X}$, we can define following two sets:

$$C_{x^*} = \{c | c \in C, x^*_c > 0\}$$

$$H_{x^*} = \{h | h \in H, x^*_h < 1\}$$

Then we can change our claim into that there exists $x' \in \mathcal{X}$ such that $C_{x'} \cap H_{x'} = \emptyset$. Suppose not, and let x' be the solution that minimize $|C_{x'} \cap H_{x'}| > 0$. In this case, we can choose following $\delta > 0$.

$$\delta = \min\left(\min_{c \in C_{x'}}\{x'_c\}, \min_{h \in H_{x'}}\{1 - x'_h\}\right)$$

Then here, we can construct following revised solution x'' by

$$x''_u = \begin{cases} x'_u - \delta & u \in H_{x'} \\ x'_u + \delta & u \in C_{x'} \\ x'_u & \text{otherwise} \end{cases}$$

It is obvious that x'' is feasible solution of LPVC. Actually, x'' is an optimal solution. First, for all vertices in $H_{x'}$ can't be incident with $C \setminus C_{x'}$. (Because, vertices in $C_{x'}$ assigned 0) It implies that, $N_G(H_{x'}) \cap C \subseteq C_{x'}$. However, we know that $|H_{x'}| \leq |N_G(H_{x'}) \cap C|$ by above lemma. So finally, we these two relation,

$$|H_{x'}| \leq |C_{x'}| \implies |x''| \leq |x'|$$

Since x' is optimal, $|x''| = |x'|$ and so x'' also optimal solution. However, at least one vertex in $H_{x'}$ or $C_{x'}$ are collapsed into 1, 0 respectively, so $|C_{x''} \cap H_{x''}| < |C_{x'} \cap H_{x'}$ which contradicts to choice of x'. So that, there exists $x' \in \mathcal{X}$ such that $C_{x'} \cap H_{x'} = \emptyset$, holds our claim. □

Remark During the proof, we construct the algorithmic way to convert any optimal LPVC solutions into half-integral optimal LPVC solution. Since we can obtain any LPVC solution in polynomial time and above algorithm also, we can find half-integral optimal LPVC solution x^* in polynomial time always.

Obtain Half-Integral Solution

For given graph $G = (V, E)$, we can obtain half-integral optimal LPVC solution x^* in polynomial time. And define

$$V_a(x^*) = \{v \in V | x_v^* = a\}$$

Then here, crown decomposition w.r.t. x^* is $(V_{0.5}, V_1, V_0)$.

Lemma 2.3. *For $G' = G[V_{0.5}]$, $\forall v \in G', x'_v = 0.5$ is an optimal solution of LPVC of G'.*

Proof. Very easy. We can observe that, on the crown decomposition of G, all $V_{0.5}$ vertices are incident with $V_{0.5}$ itself or V_1. So, if there exists another optimal solution than x', we can transplant that solution directly in G, then we can make more optimal solution to LPVC of G. This occur contradiction, so x' is an optimal solution of LPVC of $G[V_{0.5}]$. □

Lemma 2.4. *There exists an optimal vertex cover containing V_1.*

Proof. Let S be any optimal vertex cover of G. Then, w.r.t. the crown decomposition of G by $(V_{0.5}, V_1, V_0)$, we can define

$$S_0 = S \cap V_0, \quad S_{0.5} = S \cap V_{0.5}, \quad S_1 = S \cap V_1$$

Then let's construct new set

$$S' = (S \setminus S_0) \cup V_1$$

i.e., replace all cover vertices in C into remaining vertices in H. One can be easily observed, S' is a vertex cover of G. So, to check S' also an optimal vertex cover, we just need to check that number of removed vertices ($|S_0|$) is larger or equal than number of added vertices ($|V_1 \setminus S_1|$). Not too hard to see. Imagine the crown decomposition diagram. Since S was vertex cover, all the edges are covered. It means that, there aren't any edges between $V_0 \setminus S_0$ and $V_1 \setminus S_1$. So, $N(V_1 \setminus S_1) \cap V_0 \subseteq S_0$. And according to the Lemma 2.3.1.,

$$|V_1 \setminus S_1| \leq |N(V_1 \setminus S_1) \cap V_0| \leq |S_0|$$

This completes the proof. □

Lemma 2.5. *The instance $I' = (G', k')$ is equivalent to $I = (G, k)$ where $G' = G[V_{0.5}]$ and $k' = k - |V_1|$. Furthermore, it holds that $k' - lp^*(G[V_{0.5}]) = k - lp^*(G)$*

Proof. Let's prove that $I' \iff I$ first.

($\Rightarrow$) Suppose that (G', k') is Yes instance. Then, take a vertex cover S' for G' whose size is at most k'. Then, for G, just set $S = S' \cup V_1$. It is clearly vertex cover of G and

$$|S| = |S'| + |V_1| \leq k' + |V_1| = k$$

so (G, k) is Yes instance.

($\Leftarrow$) Suppose that (G, k) is Yes instance. Then, by Leamma 2.3.4. there exists an optimal vertex cover S containing V_1. Then, $S_{1/2} = S \setminus V_1 \subseteq V_{1/2}$ holds. Then, this cover all the vertices in $V_{1/2} \setminus S_{1/2}$ since if not, it is contradicts to optimal LPVC solution. So, $S_{1/2}$ is vertex cover of $G[V_{1/2}]$. However, since $|S| \leq k$,

$$|S_{1/2}| = |S| - |V_1| \leq k - |V_1| = k'$$

So, (G', k') is Yes instance.

For the further statement, we already have lemma that $lp^*(G[V_{0.5}]) = \frac{1}{2}|V_{1/2}|$. And for optimal LPVC half-integral solution, $lp^*(G) = |V_1| + \frac{1}{2}|V_{1/2}|$. So the claim holds. $\square$

Remark We show that $\forall v \in V_{1/2}, x_v^* = 1/2$ can be optimal LPVC solution for $G[V_{1/2}]$. It does not guarantee that it is unique.

Now, with Lemma 2.3.5. we can reduce(or preprocess) our original instance (G, k).

Reduction Rule of LPVC

Let (G, k) be given instance.

1. For every half-integral optimal LPVC solution y^* of (G, k),
2. If there aren't any vertex v such that $y_v^* = 1$, then end.
3. If such v exists, construct $(G[V_{1/2}(y^*)], k - |V_1(y^*)|)$

It holds because of Lemma 2.3.5.

With apply forever this rule, we finally obtain (G', k') whose optimal half-integral LPVC solution is $\forall v \in V(G'), x_v^* = 1/2$. Then, we can construct new Vertex Cover algorithm.

Algorithm 3 $\mathcal{O}(4^{k-lp^*(G)})$-time algorithm for Vertex Cover

```
procedure VC-above-LP(G, k)
    (G, k) ← Apply reduction rule until converge
    // Here, all-1/2 is the unique half-integral optimal solution.
    if k < lp*(G) = 1/2|V(G)| then
        return No                                   ▷ We observed this fact
    else if k ≥ 2 · lp*(G) = |V(G)| then
        return Yes                                  ▷ Because we can pick every vertices
    else
        uv ← Pick(E(G))
        return VC-above-LP(G − u, k − 1) ∨ VC-above-LP(G − v, k − 1)
```

It is easy to see that, this algorithm works correctly. Let's consider following measure:

$$\mu(G, k) = k - lp^*(G)$$

i.e., this is our new parameterization for Vertex Cover above LP. Then, our goal is that, by showing this factor decreases at least $\frac{1}{2}$ per each step, so our algorithm terminated and depth of tree is at most $2(k - lp^*(G))$.

Proposition 2.6. *For every iteration, the measure μ decreases at least $\frac{1}{2}$.*

Proof. Here we claim that, for every $v \in V(G)$,

$$lp^*(G - v) \geq lp^*(G) - \frac{1}{2}$$

Suppose not. Then, $lp^*(G-v) < lp^*(G) - \frac{1}{2}$. Since each lp^* value is quantumized by 1/2, it implies that

$$lp^*(G - v) \leq lp^*(G) - 1$$

Then, let's choose the half-integral optimal LPVC solution x^* of $G - v$. Then, append x^* for v with $x^*_v = 1$. Then such new x^* holds size $lp^*(G)$ optimal solution of LPVC of G, however, all-1/2 is unique solution. It is contradiction, so claim holds. With that, for every iteration, W.L.O.G. for vertex v,

$$\mu(G - v, k - 1) = k - 1 - lp^*(G - v) \leq k - lp^*(G) - \frac{1}{2} = \mu(G, k) - \frac{1}{2}$$

However, our algorithm guarantees that if $k < lp^*(G)$ or $k \geq 2 \cdot lp^*(G)$ then terminate. So, if $k - lp^*(G) < 0$, then terminate. So we can decrease our measure at most $2\mu(G, k)$ times. So the running time is $\mathcal{O}(4^{k-lp^*(G)})$. And also, we guaranteed that $k - lp^*(G) \leq k/2$. So, it works as well as $\mathcal{O}(2^k)$ time.

□

Chapter 3

Kernelization

3.1 $\mathcal{O}(k^2)$-Vertex Kernel

Kernelization

A *kernelization* of problem P is a polynomial time algorithm (w.r.t. $|x|$ and k both) such that transform any instance (x, k) of P into another instance (x', k') of P satisfying

- equivalence: $(x, k) \in P$ if and only if $(x', k') \in P$
- size bound: $|x'| \leq g(k)$ and $k' \leq g(k)$ for some computable function g.

The final instance (x', k') is called *kernel*, and $g(k)$ be the size of a kernel.

Reduction Rules

Kernelization consists in applying some package of rules. A *reduction rules* for P is a polynomial time algorithm (w.r.t. $|x|$ and k both) such that transfrom any instance (x, k) of P into another instance (x', k') of P satisfying equivalence. (Equivalence is also called by *safeness, soundness*) We say that a instance (x, k) is *irreducible* if reduction rule R does not change them.

Intuitively, if we can reduce the instance size as much as we want (bounded by $g(k)$) in polynomial time, then we can imagine that solving time of the reduced problem only depends on $g(k)$, the new input size. Following lemma is based on this intuition.

Lemma 3.1. *Let P be the parameterized decidable problem. P is fixed parameter tractable (FPT) if and only if it admits a kernelization.*

Proof. ($\Leftarrow$) Suppose that we can find kernelization:

$$(x, k) \mapsto (x', k') \quad |x'|, k' \leq g(k)$$

Then since $|x'| \leq g(k)$, we can solve this problem by brute-force in $h(g(k))$ time. So this is FPT.

($\Rightarrow$) Suppose that P is FPT. i.e., we can decide P in time $f(k) \cdot |x|^c$ time. Then, running this algorithm for time $|x|^{c+1}$. If it terminates, then return any small-sized constant instance. Otherwise, it means that $|x| < f(k)$. It directly means that given original instance is a kernel already.

□

Now, let's see the first example.

Buzz' Kernelization: $\mathcal{O}(k^2)$-Vertex Kernel

This is kernelization algorithm for VERTEX COVER. It input (G, k) and output (G', k') such that $|V(G')| \leq \mathcal{O}(k^2)$ and $k' \leq k$ with safeness. (So it is valid kernelization).

- Reduction Rule 1: $\exists v \in V(G), d_G(v) = 0 \implies (G, k) \mapsto (G - v, k)$
- Reduction Rule 2: $\exists v \in V(G), d_G(v) \geq k + 1 \implies (G, k) \mapsto (G - v, k - 1)$

It is easy to see that Rule 1 and 2 are safe. (i.e., after any reduction, (G, k) is yes instance if and only if (G', k') is yes instance.)

Now, let's analyze the size of kernel. Let (G', k') be the kernel(maybe) of above reduction rules. Then, $k' \leq k$ is clear. The measurement of size of kernel is quite tricky. We claim that if (G, k) is irreducible w.r.t. above rules, then $|G|(= |V(G)) \leq \mathcal{O}(k^2)$.

Proof. Suppose that (G, k) is yes instance. Then there exists a vertex cover C of G. Here,

$$|C| \leq k, \quad \forall v \in V(G), 1 \leq d_G(v) \leq k \implies |E(C)| \leq k^2$$

However, $V(G) \setminus C$ is independent set in G. And there aren't any isolated vertices in G, $|V(G) \setminus C| \leq |E(C)| \leq k^2$. So,

$$|G| = |V(G)| = |C| + |V(G) \setminus C| \leq k + k^2 \in \mathcal{O}(k^2)$$

Next one is tricky. If $|G| > \mathcal{O}(k^2)$, it immediately implies that (G, k) is no-instance. So, in this case, we can output tiny no-instance example for kernel output.

□

3.2 LP-based kernelization for Vertex Cover

Actually, this is sort of toy example. Now we'll see amazing results further. Before move on, let us enable to use some useful decomposition of graphs without proof. Actually, we already see this in previous chapter.

Nemhauser-Trotter Theorem

Given any graph G, we can find partition $V(G) = (R, H, C)$ holds followings in polynomial time.

- For any vertex cover S_r of $G[R]$, $S_r \cup H$ is a vertex cover of G.
- There exists an optimal vertex cover containing H.
- Any vertex cover of $G[R]$ is of size at least $\frac{1}{2}|R|$.

Then, let's define kernelization related to this theorem.

$\mathcal{O}(2k)$-kernelization of Vertex Cover

This is kernelization algorithm for Vertex Cover. It takes input (G, k) and output (G', k') such that $|V(G)| \leq 2k$.

- Rule 1: $V(G) \mapsto (R, H, C)$, $H \cup C \neq \emptyset \implies (G, k) \mapsto (G[R], k - |H|)$

Lemma 3.1. *This holds safeness.*

Proof. ($\Leftarrow$) Let S' be a vertex cover of $G[R]$ whose size is at most $k - |H|$. Then, by the condition 1 of NT theorem, $S = S' \cup H$ be the vertex cover of G whose size is at most k.

($\Rightarrow$) Let S be a vertex cover of G whose size is at most k. Then, there exists a vertex cover S of G, such that $H \subseteq S$. One can be easily derived from condition 1: C is independent set of G and R does not incident with C. First one is easy, and for second one, suppose that $cr \in E(G)$ for $c \in C, r \in R$. Then, we can choose $R - r$ for the vertex cover of $G[R]$ obviously. However, since $H \cup (R - r)$ does not cover cr, it is contradiction. So, R does not incident with C. So that, H cover all the incident edges of C. Since S is optimal, $S \setminus H$ is vertex cover of $G[R]$ whose size is at most $k - |H|$.

□

Lemma 3.2. *The kernel size($= |V(G)|$) of above kernelization is $2k$.*

Proof. Let (G, k) be irreducible instance of above algorithm. Then, it implies that for NT decomposition (R, H, C) of G, $H \cup C = \emptyset$. Then, it means that $G[R] = G$. So, by condition 3, any vertex cover of $G[R] = G$ is size at least $\frac{1}{2}|R| = \frac{1}{2}|V|$. So, if $k < \frac{1}{2}|V|$, it automatically becomes no instance. So we can output tiny no-instance kernel for this case, $|V| > 2k$. Otherwise, $|V| \leq 2k$. So kernel size of (G, k) is less than $2k$.

□

3.3 $3k$-vertex kernel using Crown Decomposition

Crown Decomposition

A *crown decomposition* of a graph $G = (V, E)$ is a partition (R, H, C) of V s.t.

- $H \cup C \neq \emptyset$
- C is independent set of G
- $N(C) \subseteq H$, i.e., H seperates C and R.
- $\forall H' \subseteq H$, $|N(H') \cap C| \geq |H'|$, i.e., Hall's condition holds between H, C

Matching and Saturated

A *matching* in a graph $G = (V, E)$ is a set of disjoint edges. We call that M is maximal matching when if we add any edge to M, it is not matching anymore. We call that M is maximum matching when M is the largest matching. We say that $v \in V(G)$ is *saturated* by matching M if v is incident with M.

We have two famous theorems in Graph Theory course, which are Hall's theorem and könig's theorem.

Hall's Theorem

Let G be a bipartite graph partited as (X, Y). There exists a matching M such that saturate all vertices in X if and only of $\forall X' \subseteq X, |N(X') \cap Y| \geq |X'|$ holds.

König's Theorem

Vertex covering and Matching is dual in bipartite graphs. i.e., the size of optimal vertex cover and maximum matching is same on the bipartite graph.

From now on, we'll explore various properties on crown decomposition and use it to construct kernelization.

Lemma

If (R, H, C) is a crown decomposition of G, then there exists an optimal vertex cover of G containing all vertices of H.

Proof. Suppose that S be the any vertex cover of G, whose crown decomposition is (R, H, C). Then, w.r.t. such decomposition, we can also decompose our vertex cover S into (S_R, S_H, S_C). Note that, $E(H \setminus S_H, C \setminus S_C) = \emptyset$. So, it means that, $N(H \setminus S_H) \cap C \subseteq S_C$. Then, $|N(H \setminus S_H) \cap C| \leq |S_C|$. By Hall's condition,

$$|H \setminus S_H| \leq |N(H \setminus S_H) \cap C| \leq |S_C|$$

And it is clear that $S' = S - S_C + (H \setminus S_H)$ is a vertex cover. Since

$$|S'| = |S| - |S_C| + |H \setminus S_H| \leq |S|$$

holds, it implies that there exists an optimal vertex cover containing entire H.

□

Similar to previous chapter, this lemma gives us following reduction rule.

Reduction Rule 1

For input (G, k), let the crown decomposition of G be (R, H, C). Then, output $(G - (H \cup C), k - |H|)$ i.e.,

$$(G, k) \mapsto (G - (H \cup C), k - |H|)$$

Now, we can prove that this is safe easily.

Proof. If (G, k) is yes instance, then by above lemma, there exists optimal vertex cover S containing entire H. Then, it is clear that $S_C = \emptyset$ and S_R is optimal vertex cover of $G[R]$. So, $(G - (H \cup C), k - |H|) = (G[R], k - |H|)$ is also yes.

Conversely, obvious.

□

Actually, you might realized that this reduction rules is same to previous section's one. What's the difference? Previous one is LP-based decomposition. As we've seen, it also generate crown decomposition. However, what we defined for crown decomposition is more general concept. So, one can expect is that crown decomposition algorithm is much faster than construct crown decomposition via solving half-integral LP. So we need to construct algorithm for general crown decomposition.

To show how can we construct crown decomposition and kernelization by Reduction Rule 1 at once, first we propose the complete kernelization algorithm. (See next page)

Algorithm 4 $4k+1$ Kernelization for VERTEX COVER using crown decomposition

```
procedure VC-CROWN(G, k)
    Remove all isolated vertices in G
    if |V(G)| ≤ 4k + 1 then return (G, k)                ▷ Kernelization Success
    else
        M ← Find a maximal matching via greedy            ▷ Just pick until can't
        if |M| ≥ k + 1 then return small No-instance (G_0, k_0)
            C_0 ← V(G) − V(M), H_0 ← N(C_0), R_0 = V(G) − (C_0 ∪ H_0)
            while ∃H' ⊆ H_i s.t. |H'| > |N(H') ∩ C_i| do      ▷ (*)
                C_{i+1} ← C_i − (N(H') ∩ C_i)
                H_{i+1} ← H_i − H'
                R_{i+1} ← R_i ∪ H' ∪ (N(H') ∩ C_i)
                i ← i + 1
            return VC-CROWN(G − (H_i ∪ C_i), k − |H_i|)
```

Here, suppose that (*) works well, i.e., it terminates in polynomial time and always return crown decomposition of G. Then, the safeness of this kernelization is proved. Here, what we need to show to claim this is valid kernelization is that

Crown Decomposition Algorithm

- the result of while loop is crown decomposition,
- checking of while condition (*) can be done in polynomial time.

Proof. Let's see the first one. Here, we claim loop invariants first;

1. $H_i \cup C_i \neq \emptyset$
2. C_i is independent set of G
3. $N(C_i) \subseteq H_i$, i.e., H_i seperates C_i and R_i
4. $|H_i| < |C_i|$

- (Initial Case)
    1. $H_0 = N(C_0)$. So, if $H_0 \cup C_0 = \emptyset$, it implies that $C = \emptyset$. It means that M is perfect matching of G. However, $|V(G)| > 4k+2$, so if the perfect matching exists, $|M| > 2k+1$, contradiction. So, $H_0 \cup C_0 \neq \emptyset$.
    2. Since M is maximal matching, $C_0 = V(G) - V(M)$ must be independent.
    3. Since $H_0 = N(C_0)$, this is obvious.
    4. We know that $|V(G)| > 4k+2$ and $|M| \leq k$, so $|V(M)| \leq 2k$, so $|C_0| > 2k+2$. However, $H_0 = N(C_0) \subseteq V(M)$ since if not, contradiction to M is maximal. So, $|H_0| \leq |V(M)| \leq 2k$. It implies that $|H_0| < |C_0|$ holds.

- (Induction Step) It is recommended : Draw how we derive next iteration.

  1. Since $|H_i| < |C_i|$, the entire H_i never selected to H'. So, $H' \subsetneq H_i$, so $H_{i+1} \cup C_{i+1} \neq \emptyset$.
  2. $C_{i+1} \subset C_i$, so C_{i+1} is obviously independent set.
  3. It is sufficient to show that C_{i+1} and $H' \cup (N(H') \cap C_i)$ is independent. It is clear that C_{i+1} and $N(H') \cap C_i$ is independent, since C_i is independent. And, $N(H') \cap C_{i+1} = \emptyset$ since we removed all neighbors of H' in C_i.
  4. $|H_i| < |C_i|$. For $|H_{i+1}|$, we removed $|H'|$ vertices. For $|C_{i+1}|$, we removed $|N(H') \cap C_i|$ vertices. Since $|H'| > |N(H') \cap C_i|$, automatically $|H_{i+1}| < |C_{i+1}|$ holds.

It implies that, if this while loop terminate, (1)-(3) holds and no such H' exists, so it becomes crown decomposition of G.

Now, let's see the second one. Which states we can decide such H' (if exists) in polynomial time. Here, there is a polynomial time algorithm for Minimum Vertex Cover problem on bipartite graph. Let's imagine bipartite graph $(H \cup C, E(H, C))$. And apply above algorithm here, then we can find minimum vertex cover X.

If $H_i \subseteq X$, then implies $H_i = X$. Then there exists matching M^* such that saturate all vertices in H_i. So, Hall's condition holds. Then we can't find H'.

If $H_i \setminus X \neq \emptyset$, then let $H' = H_i \setminus X$. Here, $N(H') \subseteq X \cap C_i$. If $|X \cap C_i| > |H'|$, then we can find minimum vertex cover by alternating all vertices in $X \cap C_i$ into H'. So, $|H'| \leq |X \cap C_i|$. If same, we can imagine that $H_i = X$ case. If else, our choice is good.

All these processes can be done in polynomial time without brute forces.

$\square$

3.4 $\mathcal{O}(k^2)$-Vertex Kernel for Feedback Vertex Set

This section require quite long proofs and theorems. So, I'll write the full algorithm first. Note that, in this problem we assume that G **allows multigraphs even loops.**

Algorithm 5 $\mathcal{O}(k^2)$-kernelization for Feedback Vertex Set

```
procedure FVS-Kernel(G, k)
    if k < 0 then return (G_0, k_0)                          ▷ Tiny No instance
    (G, k) ← Closure under Reduction Rule 1-4.
    if |V(G)| ≤ 7k^2 + 7k then return (G, k)                 ▷ Kernelization Success
    else if Δ(G) < 7k then return (G_0, k_0)                 ▷ Tiny No instance
    else
        x ← x ∈ V(G), d_G(x) ≥ 7k
        if x has k + 1 size x-flower then return (G − x, k − 1)   ▷ Reduction Rule 5
        else
            Assert : ∃S_x ⊆ V(G) − x, hitting all x-cycles, |S_x| ≤ 2k
            C ← all components of G − S_x − x, connected with x
            if C has more than k cycles then return (G_0, k_0)
            else
                T ← Tree components in C joined with x by unique edge.
                S', T' ← apply 2-expansion Lemma with S_x, T
                return (G, k) ← Apply reduction rule 6 with S', T'
```

Reduction Rules for FVS kernelization

1. If $x \in V(G)$ has loops, then $(G - x, k - 1)$
2. If $x \in V(G), d_G(x) = 0, 1$, then $(G - x, k)$
3. If $u, v \in V(G)$ is connected by more than 2 parallel edges, remaining only 2.
4. If $x \in V(G), d_G(x) = 2, ux, vx \in E(G)$ then $(G - x + uv, k)$
5. If $x \in V(G)$, x has $k + 1$-star, then $(G - x, k - 1)$
6. Suppose that there exists $x \in V(G)$, $S \subseteq V(G) - x$, $\mathcal{T}$ is subset of components in $G - S - x$ such that
   - $\forall \mathcal{C} \in \mathcal{T}, G[\mathcal{C}]$ is tree.
   - $\forall \mathcal{C} \in \mathcal{T}$, x and $\mathcal{C}$ is connected by unique edge.
   - $\forall Z \subseteq S$, at least $2|Z|$ components in $\mathcal{T}$ is connected with Z.

   Then, we obtain new kernel (G', k') by

- Remove all unique edges between x and $\mathcal{T}$.
- Make x and all vertices in S connected via double edges.
- $k' = k$

Yes, this needs quite long context of proof. Let's do this step-by-step.

Lemma 3.1. *Reduction Rules 1-4 is safe and we can find irreducible kernel (G, k) in polynomial time. Then, $\delta(G) \geq 3$ holds.*

Lemma 3.2. *Suppose that (G, k) be the above kernel(RR 1-4). If (G, k) is Yes-instance of* Feedback Vertex Set*, then following always holds:*

$$\Delta(G) < d \implies |V| < (d+1)k, \quad |E| < 2dk$$

In other words, if it is vanished, (G, k) is always No-instance.

Proof. Since (G, k) is Yes instance and kernel of RR 1-4, we can obtain

$$\exists X \subseteq V(G), \quad |X| \leq k, \quad G - X \text{ is forest }, \quad 3 \leq d_G(v) < d$$

Then,

$$\begin{aligned} 3|V - X| \leq \sum_{u \in G-X} d_G(u) &= \big|E(X, G-X)\big| + 2\big|E(G-X)\big| \\ &\leq dk + 2(|V - X| - 1) && \text{(Remark 1)} \\ &\leq 2|V| + (d-2)k - 2 \\ &< 2|V| + (d-2)k \end{aligned}$$

This implies that

$$3|V| - 3k < 2|V| + (d-2)k \implies |V| < (d+1)k$$

For the edges,

$$\begin{aligned} |E| &= \text{edges incident with } X + \text{edges does not touch } X \\ &\leq dk + |V - X| - 1 \\ &< 2dk && \text{(By Remark 1)} \end{aligned}$$

□

With these, we can run our algorithm until line 7. If size is bounded, then return that as a kernel. If size is unbounded by $d = 7k$ into above lemma, it vanishes Lemma 3.4.2. directly. So (G, k) is automatically No instance, so output tiny No-instance here. Now we arrive at line 7. Let's do further analysis with added condition:

$$3 \leq \delta(G), 7k \leq \Delta(G), |V| > (7k+1)k$$

x-cycle and x-flower

x-cycle is a cycle such that containing x. *x-flower* is collection of x-cycles such that vertex disjoint except only x. The *order* of an x-cycle is number of x-cycles in that.

With this notion, it is easy to show that line 8 is safe and run in polynomial time with respect to n.

Lemma 3.3. *Reduction Rule 5 is safe and run in polynomial time.*

Then now, let's move on to line 10. Here we have that x does not have $k+1$ size flower and meet an assertation:

$$\exists S_x \subseteq V(G) - x, \text{ hitting all } x\text{-cycles}, |S_x| \leq 2k$$

We need to prove that this can be asserted always.

Want To Show

We want to show that, at line 7, one of following must hold:

- x has $k+1$ size of x-flower.
- No more than k flower, then $\exists S_x \subseteq V(G) - x$, hitting all x-cycles, $|S_x| \leq 2k$

A-path

For $A \subseteq V(G)$, we say that a path P is A-path if $|P| \leq 1$ and only endpoints of P are contained in A.

We can divide the neighbors of x into 2 groups:

- A_1: Neighbors of x, connected with x by one edge.
- A_2: Neighbors of x, connected with x by two edges.

In this setting and notion of A-paths, we can construct following lemma.

Lemma 3.4. *Let (G, k) be any graph and $x \in V(G)$. Following holds.*

- *Maximum order of x-flower is summation of $|A_2|$ and maximum number of vertex disjoint A_1 paths in $G - x - A_2$.*
- *$S_x \subseteq V(G) - x$ hit all x-cycles if and only if S_x contains A_2 entirely, and $S_x - A_2$ hit all A_1 paths in $G - x - A_2$.*

Proof. Easy

□

Since we can find A_2 directly, our problem is reduced as following.

Want To Show - Reduced Version

Let $l := k - |A_2|$, $A := A_1$, $H = G - x - A_2$.

- There exists $l + 1$ vertex disjoint A-paths in H.
- No more than l vertex disjoint A-paths, then $\exists S_x \subseteq V(H), S_x$ hit all the A-paths in H, and $|S_x| \leq 2l$.

We'll take Gallai's theorem in Graph Theory.

Gallai's A-path Theorem

Let $G = (V, E)$ and $A \subseteq V$. Let's denote the maximum number of vertex disjoint A-path in G by M_A.

$$M_A = \min_{X \subseteq V} \left(|X| + \sum_{C \in \mathcal{C}(G-X)} \lfloor \frac{A \cap C}{2} \rfloor \right)$$

$\mathcal{C}(G-X)$ is collection of all components in $G-X$. And this theorem is algorithmic. i.e., we can find the solution X in polynomial time.

With this theorem, we can prove our goal in red box.

Lemma 3.5. *Let $G = (V, E)$ be a graph. If there exists at most l vertex disjoint A-paths, then we can find Z such that hit all the A-paths and $|Z| \leq 2l$.*

Proof. According to the Gallai's Theorem, we can find $X \subseteq V(G)$ such that

$$|X| + \sum_{C \in \mathcal{C}(G-X)} \lfloor \frac{|A \cap C|}{2} \rfloor \leq l$$

Now, we'll construct Z from here. First, add all vertices in X into Z. And then, for every component C in $\mathcal{C}(G - X)$, choose all A-vertices except at most one. Then such cardinality of chosen vertices is same with $2 \cdot \lfloor \frac{|A \cap C|}{2} \rfloor$. And then add them into Z. Then,

$$|Z| = |X| + 2 \sum_{C \in \mathcal{C}(G-X)} \lfloor \frac{|A \cap C|}{2} \rfloor \leq 2 \left(|X| + \sum_{C \in \mathcal{C}(G-X)} \lfloor \frac{|A \cap C|}{2} \rfloor \right) \leq 2l$$

Here, this Z hits all A-paths. Since we choose quite amount of A-vertices, sufficient to consider remaining A-vertices and their paths. Remaining vertices are located in each components, at most one in there. So, if there exists any A-path, such path must through X and reach to another component. Since $X \subseteq Z$, it is hit by Z. $\square$

Now we arrive at line 11 safely. The next step will be reduction of given graph with S_x where $S_x \subseteq V(G) - x$ hitting all x-cycles and $|S_x| \leq 2k$. There remains most treaky part here. Let's rewrite Reduction Rule 6 again.

Reduction Rule 6

Apply condition: there exists $x \in V(G), S \subseteq V - x$ and $\mathcal{T} \subseteq \mathcal{C}(G - x - S)$ s.t.

- $G[C]$ is tree, for each $C \in \mathcal{T}$
- There exists unique edge between x and each C
- $\forall Z \subseteq S$, number of incident components in $\mathcal{T}$ with Z is more than $2|Z|$

Then, reduce the instance G by:

- Remove all edges between x and $\mathcal{T}$
- Make every vertices in S is adjacent with x via exactly 2 edges.

Lemma 3.6. *Reduction Rule 6 is safe.*

Proof. since we changes only the edges incident with x, only x-cycles are matter.

($\Rightarrow$) Suppose that (G, k) is Yes instance of FVS. Here, let X be an optimal FVS of G. If $x \in X$, since x hit all the newly generated x-cycles, so X is also a FVS of G'. If $x \notin X$, we'll find another optimal FVS X^* such that $S \subset X$. If we can find, then since S hit all the newly generated x-cycles in G', X^* become FVS of G'. Let $Z = S \setminus X \neq \emptyset$. For each $z \in Z$, at least 2 components $(\mathcal{T}_1, \mathcal{T}_2)$ are incident with z in $\mathcal{T}$ (Apply Condition 3) Then since we can find $z \to \mathcal{T}_1 \to x \to \mathcal{T}_2$ cycle, X must hit this in $\mathcal{T}_1$ or $\mathcal{T}_2$. Let's remove all the vertices of X in $\mathcal{T}$ and add all vertices in Z.

$$X^* = \left(X \setminus \bigcup_{T \in \mathcal{T}} V(T) \right) \cup Z$$

Here, since amount of neighbor components of Z is larger than $2|Z|$, at least $|Z|$ vertices of $|X|$ are included in $\mathcal{T}$. Since we remove all them and add Z, $|X^*| \leq |X|$. And X^* is clearly FVS since all cycles s.t. vertices in $\mathcal{T}$ covered were hit also vertices in Z. So Z hit all the cycles that $X \setminus \bigcup_{T \in \mathcal{T}} V(T)$ covered, so X^* is FVS. Then X^* is also FVS of G'.

($\Leftarrow$) Here, for any FVS X' of G', $x \in X'$ or $S \subseteq X'$. So both case, optimal FVS X' of G' also be the FVS of G.

$\square$

One we must show is that, we can apply Reduction Rule with conditions until line 14 always. Actually, it is possible.

Lemma 3.7. *Let H be a bipartite graph $S \uplus T$. In polynomial time, one can find a set $Z \subset S$ with $|N(Z)| < 2|Z|$ if such Z exists.*

Proof. Using vertex copy of S then König's theorem and obtain maximum matching. □

Lemma 3.8. *Let H be a bipartite graph with a bipartition $S \uplus T$ s.t.*

- *every vertex of T adjacent with vertex of S*
- $|T| \geq 2|S|$

Then, in polynomial time one can find vertex sets $S' \subseteq S$ and $T' \subseteq T$ such that

- $N(T') = S'$ *and*
- $|N(Z)| \geq 2|Z|$ *for every* $Z \subseteq S'$

Lemma 3.9. *Line 14-16 is always applicable.*

Proof. We have $S_x \subseteq V(G) - x$ such that $|S_x| \leq 2k$ and S_x hits all x-cycles. However, $d_G(x) \geq 7k$. Since S_x contains all double edges neighbors of x, but there limit is k since no order $k+1$ x-flower condition. Implies that, at least $4k$ are not in S_x. Moreover, since they have not to construct any x-cycles, each of them are incident with tree components in $G - x - S_x$ via unique edges. Here, by imagine that such tree components are vertices, and S, they forms bipartite graph. Note that $|\mathcal{T}| \geq 2|S_x|$ and every $T \in \mathcal{T}$ is adjacent with at least one vertex of S, because in G, T does not have leaf. So we can apply Lemma 3.4.8, and obtain $S' \subseteq S_x$, $\mathcal{T}' \subseteq \mathcal{T}$. They holds the Apply Condition of Reduction Rule 6.

□

Chapter 4

Iterative Compression

4.1 $\mathcal{O}(5^k)$-time algorithm for Feedback Vertex Set

Compression FVS

Compression FVS
Input: Graph $G = (V, E)$ and Feedback Vertex Set X of G.
Parameter: $|X|$
Output(Decision): Does G have another FVS Y such that $|Y| < |X|$?

The Iterative Compression method is that, solve original problem via solving its compression version. In this case, we'll study Compression version of Feedback Vertex Set problem. Suppose that our graph is evolving by one vertex.

$$G_1 \to G_2 \to \cdots \to G_n = G$$

If we know X_i, the Feedback Vertex Set of G_i, then obviously $X_{i+1} = X_i \cup \{v_{i+1}\}$ become feedback vertex set of G_{i+1}. If it doesn't exceed our original budget, it can be the instance for G_{i+1}. However, if it exceed our budget, we need to solve Compression FVS(G_{i+1}, X_{i+1}). This is sort of dynamic programming, let's formalize this method based on FVS problem.

Lemma 4.1. *If we can solve* Compression FVS(G, X) *in time* $c^{|X|} \cdot n^d$*, then we can solve* Feedback Vertex Set(G, k) *in time* $c^k \cdot n^{d+1}$*.*

Proof. Let's enumerate vertices of G by

$$v_1, v_2, \cdots, v_n, \quad G_i := G[\{v_1, \cdots, v_i\}]$$

We construct algorithm that we mentioned above. The worst case running time of Compression FVS(G, X) is $c^k \cdot n^d$ since $|X| \leq k$ in our setting. So, the worst case running time of our algorithm is that

$$n \times (c^k \cdot n^d) = c^k \cdot n^{d+1}$$

□

Thanks to this lemma, we are sufficient to show that we can solve Compression FVS in such time complexity. In similar way, we'll reduce Compression FVS again into another easier problem.

Disjoint FVS

Disjoint FVS
Input: Graph $G = (V, E)$, X is a feedback vertex set of G, $k \in \mathbb{Z}$
Parameter: $k + |\mathcal{C}(G[X])|$
Output: Does G have disjoint FVS Y (i.e., $X \cap Y = \emptyset$) s.t. $|Y| \leq k$?

Algorithm 6 Algorithm for Disjoint FVS

1: **procedure** Disjoint FVS(G, X, k)
2: $G \leftarrow$ delete all $d_G(v) = 0, 1$ vertices
3: $F \leftarrow G[V \setminus X]$, $\mathcal{C} \leftarrow$ components of $G[X]$ ▷ F is forest
4: $\forall x \in F, d_G(x) = 2, |N(x) \cap F| = 1$ then bypass x.
5: **if** G is acyclic **then return** Yes, by witness : $\emptyset$
6: **if** $G[X]$ contains any cycle **then return** No
7: **if** G has any cycle and $k <= 0$ **then return** No
8: $l \leftarrow$ any leaf vertex in F
9: **if** $\exists C \in \mathcal{C}$, l incident C with more than 2 edges **then**
10: **return** Disjoint FVS$(G - l, X, k - 1) \cup \{l\}$
11: **else if** No such cases **then**
12: **return** Disjoint FVS$(G - l, X, k - 1) \cup \{l\} \vee$ Disjoint FVS$(G, X \cup \{l\}, k)$

Lemma 4.2. *This algorithm is correctly solves* Disjoint FVS *in time* $\mathcal{O}^*(2^{\mu(I)})$ *where* $\mu(I) = k + |\mathcal{C}(G[X])|$

Proof. Since our goal is find feedback vertex set disjoint with X, line 5,6,7 is clearly correct. And for reduction step, if such x in line 4 exists, then when we need to pick x, then instead pick its F-neighbor, so this also clear. Then suppose that l is leaf in F. If line 9 occurs, then l joined in a cycle with such $\mathcal{C}$. So we must pick l as new FVS. If not occurs, we must consider 2 cases each: Y contains l or not. This is line 12.

For the running time analysis,

$$\mu(I = (G, X, k)) = k + |\mathcal{C}(G[X])|$$

For each step, if we choose $I' := (G - l, X, k - 1)$ then $\mu(I') = \mu(I) - 1$ so decreases, if we choose $I' := (G, X \cup \{l\}, k)$, since in line 12, l works as 'bridge' of components of $G[X]$ so $\mu(I') \leq \mu(I) - 1$. So our parameter always strictly decreases at least 1, and the worst branch cases are 2, so the running time would be $\mathcal{O}^*(2^{\mu(I)})$. This completes proof.

□

Lemma 4.3. *With above algorithms, we can solve* Compression FVS *in time* $5^{|X|} \cdot n^d$

Algorithm 7 Algorithm for Compression FVS

```
procedure Compression FVS(G, X)
    for ∀I ⊊ X do
        G̃ ← G − I, X̃ ← X \ I, k̃ ← |X̃| − 1
        if Disjoint FVS(G̃, X̃, k̃) returns No then Continue
        else
            Ỹ ← Disjoint FVS(G̃, X̃, k̃)
            return Yes with witness: I ∪ Ỹ
    return No
```

Proof. It is obvious that, if this algorithm returns Yes, then such instance $I \cup \tilde{Y}$ is actually smaller size FVS of G. This is because we fix I, and then find alternatives for $X \setminus I$ vertices to hit all cycles that $X \setminus I$ hit. So this algorithm is complete. How about soundness? Suppose that this algorithm returns No. If the instance (G, X) is actually Yes-instance, then there exists smaller FVS $Y \neq X$ of G. Then, we can find $Y \cap X$. Since I iterate all proper subsets of X, $I = X \cap Y$ is iterated. Since it doesn't return Yes, so Disjoint FVS$(G - X \cap Y, X - Y, |X - Y| - 1)$ returns No. However, we have remaining parts of Y, which is $Y - X$ such that $|Y - X| < |X - Y|$. Since $Y - X$ is FVS of $G - X \cap Y$, this is contradiction to Disjoint FVS is correct. So above algorithm is correct.

Finally, let's analyze the running time of this algorithm. For each iteration of I,

$$\mu(G - I, X \setminus I, |X \setminus I| - 1) = |X \setminus I| - 1 + |\mathcal{C}(\tilde{G}[\tilde{X}])| \leq 2(|X| - i) - 1$$

So, for the selected subset size i, running time will be $\mathcal{O}^*(2^{2(|X|-i)-1}) = \mathcal{O}^*(4^{|X|-i})$ So, running time will be

$$\sum_{i=0}^{|X|-1} \binom{|X|}{i} \cdot \mathcal{O}^*(4^{|X|-i}) \leq (4+1)^{|X|} \cdot n^d$$

□

Chapter 5

Randomized Algorithms

Chapter 6

Tree Decomposition

Tree Decomposition

Let G be a graph. A *tree decomposition* of G is pair (T, χ) where T is a tree and $\chi : V(T) \to 2^{V(G)}$. For each $t \in V(T)$, we say $\chi(t) \subseteq V(G)$ be a *bag* and (T, χ) satisfy following.

1. $\bigcup_{t \in V(T)} \chi(t) = V(G)$
2. $\forall e = uv \in E(G), \quad \exists\, t \in V(T),\ \{u, v\} \subseteq \chi(t)$
3. $\forall v \in V(G)$, $\{t \in V(T) \mid v \in \chi(t)\}$ forms a subtree of T.

Note that, this definition is somewhat similar to *topology* on sets. We define the measure that how finer our tree decomposition is, called *width*.

Definition 6.1. For each tree decomposition (T, χ) of G, we define *width* by $\max_{t \in V(T)} |\chi(t)| - 1$. i.e., the largest bag size - 1. And we denote the global measure of G by *treewidth* of G, defined by

$$tw(G) = \min_{(T,\chi)} \operatorname{width}(T, \chi)$$

Definition 6.2. Suppose that $A, B \subseteq V(G)$. We define (A, B)-path in G as a path with one endpoint is in A and another is in B. Note that $v \in A \cap B$ is trivial (A, B)-path. We call that X is an (A, B)-seperator in G if every (A, B)-path contains a vertex of X. We define *contraction* of G by $e = uv \in E(G)$ is an graph modification such that merge u, v. Formally, delete u, v and add new vertex z_{uv} and then connect with $N_G(u) \cup N_G(v)$. Denote such modified graph by G/e.

Propositions

(1) $tw(G) - 1 \leq tw(G - v) \leq tw(G)$

(2) $tw(G) - 1 \leq tw(G - e) \leq tw(G)$

(3) $tw(G) - 1 \leq tw(G/e) \leq tw(G)$

(4) G is a forest $\iff$ $tw(G) \leq 1$

Nice Tree Decomposition

A nice tree decomposition (T, χ) of G is a rooted tree decomposition such that each $t \in V(T)$ is one of the following types:

- Leaf node: t is a leaf node of T
- Introduce node: t has exactly one child t' holds that $\chi(t) = \chi(t') \cup \{v\}$ for some $v \in V(G)$.
- Forget node: t has exactly one child t' and holds that $\chi(t) = \chi(t') \setminus \{v\}$ for some $v \in V(G)$.
- Join node: t has exactly 2 children t_1, t_2 such that $\chi(t) = \chi(t_1) = \chi(t_2)$.

Lemma 6.3. *Let (T, χ) be a tree decomposition of G of width w. In linear time, we transform this into a nice tree decomposition (T', χ') of G of width w. Moreover,*

- *Root node and Leaf nodes are empty bags. (If we want)*
- $|V(T')| \leq 4wn$.

Proof. Let root be empty bag and for every leaf node of T we append empty bag. Choose any $t \in V(T)$. Following that, if t has d children then we construct d-times of join nodes and connect each children into unique child of generated join nodes. If t has unique child, remove $\chi(t) \setminus \chi(t')$ one-by-one via introduce nodes and add $\chi(t') \setminus \chi(t)$ one-by-one via forget nodes. □

Definition 6.4. We define that T_t be the subtree of T rooted at t.

$$V_t = \bigcup_{b \in V(T_t)} \chi(b) \quad G_t = G[V_t]$$

Lemma

Let (T, χ) be a nice tree decomposition of G. Suppose that $t \in V(T)$ is an introduce node with its unique child t' such that $\chi(t) = \chi(t') \cup \{v\}$ for some $v \in V(G)$. Then,

$$N_{G_t}(v) \subseteq \chi(t)$$

Proof. T_t become a tree decomposition of G_t. However, the root is unique node of tree decomposition such that containing v, so every edges incident with v in G_t is contained in t. This exactly implies $N_{G_t}(v) \subseteq \chi(t)$ □

Lemma

Let (T, χ) be a nice tree decomposition of G. Suppose that $t \in V(T)$ is an join node with t_1, t_2 then $V_{t_1} \cap V_{t_2} = \chi(t)$.

Proof. Note that vertices appears in $V_{t_1} \cap V_{t_2}$ must be contained in $\chi(t)$ since (T, χ) is a tree decomposition. □

Maximum Independent Set

For a graph $G = (V, E)$, find a maximum size of independent set of G.

Theorem 6.5. *There is an algorithm which, given G and non-redundant tree decomposition (T, χ) such that width $\leq w$, finds a maximum size of independent set of G in time $\mathcal{O}(w2^w \cdot n)$.*

Step 0

We first transform (T, χ) into a *nice tree decomposition* via Lemma 6.3.

Step 1

Define following for $t \in V(T)$ and $S \subseteq \chi(t)$:

$$\mathrm{IND}_t[S] = \begin{cases} \text{Size of maximum independent set } I \text{ of } G_t \text{ such that } I \cap \chi(t) = S \\ \perp \quad \text{if no such } I \text{ exists} \end{cases}$$

Remark that Maximum size of independent set of G is $\mathrm{IND}_r[\emptyset]$.

Planning Mode

(Initialization) At each leaf t of T, compute a basis cases for DP.

$$\Downarrow$$

(Recurrence) For forget/introduce/join node t, define recursive relation

$$\Downarrow$$

(Runtime efficiency) Compute $\mathrm{IND}_t[S]$ for each $S \subseteq \chi(t)$ efficiently.

Remark 6.6. Note that our algorithm answer with non-witness i.e., does not compute actual maximum independent set.

Initialization

Let t be any leaf node of T. For every $S \subseteq \chi(t)$, we can easily compute

$$\mathrm{IND}_t[S] = \begin{cases} |S| & S \text{ is an independent set} \\ \perp & \text{otherwise} \end{cases}$$

Recurrence

We construct following relations for each nodes' type.

If t is a forget node, then the child t' holds that $\chi(t) = \chi(t') \setminus \{v\}$. For $S \subseteq \chi(t)$,